

Fraud Detection Using Graph Neural Networks: A Relational Learning Approach with Explainability

Mr. Pradeep Upadhyay
Professor, Dept of CSE, IILM University
Greater Noida, India
pradeep.upadhyay@iilm.edu

Utkarsh Kishor
IILM University
Greater Noida, India
utkarsh.kishor.cs27@iilm.edu

Keshav Kumar
IILM University
Greater Noida, India
keshav.kumar.cs27@iilm.edu

Harjit Singh Bhadauriya
IILM University
Greater Noida, India
harjit.bhadauriya.cs27@iilm.edu

Anurag Kumar Upadhyay
IILM University
Greater Noida, India
anurag.kumar.cs27@iilm.edu

Aurk Mishra
IILM University
Greater Noida, India
aurk.mishra.cs27@iilm.edu

Abstract—Economic and Financial fraud remains one of the most economically devastating challenges confronting global digital ecosystems, with estimated annual losses exceeding USD 5 trillion. Classical machine learning approaches—logistic regression, gradientboosted trees, and multilayer perceptrons (MLPs)—fail to model the structural and relational patterns of sophisticated fraud schemes because they treat every transaction as an isolated, independent feature vector. Fraud rings, collusive account clusters, and card-sharing networks only become detectable when transactions are analysed as an interconnected graph. This paper presents the design, implementation, and evaluation of a Heterogeneous Graph Attention Network (HeteroGAT) for financial fraud detection, built upon the IEEE-CIS Transaction Fraud dataset comprising 590,540 e-commerce transactions. The system constructs a heterogeneous graph with five node types—Transaction, Card, Email, Device, and Address—connected by semantically distinct edge relations. This multi-relational structure enables the GNN to propagate fraud signals across shared entities, catching fraud rings through 2–3-hop neighbourhood aggregation. The HeteroGAT architecture employs three layers of HeteroConv-wrapped GATConv operations with four attention heads, Batch Normalisation, ReLU activation, and Dropout ($p=0.3$), trained with class-weighted Cross-Entropy Loss (fraud weight 14.3) to address severe 3.5% class imbalance. A GNNExplainer-based explainability module and gradient-importance attribution provide transaction-level explanations and global feature rankings. Comparative baselines (MLP, GCN, GraphSAGE, GAT) are included, with HeteroGAT achieving a test AUC-ROC of 0.971 and fraud-class F1-Score of 0.86. **Index Terms**—Fraud Detection, Graph Neural Networks, Heterogeneous Graph, Graph Attention Network, Message Passing, PyTorch Geometric, GNNExplainer, Class Imbalance, IEEE-CIS, HeteroConv, GATConv, AUC-ROC

Index Terms—component, formatting, style, styling, insert

I. INTRODUCTION

Financial fraud constitutes one of the most strategically adaptive and economically significant threats facing modern digital financial infrastructure. According to the ACFE 2024 Report to the Nations, organisations globally lose an estimated 5% of revenue annually to fraud, with a median

case loss of USD 145,000. The rapid expansion of digital payment networks, open banking APIs, buy-now-pay-later (BNPL) services, and cryptocurrency ecosystems has dramatically extended the attack surface exploitable by fraud actors, rendering traditional threshold-based and rule-based detection systems obsolete. Classical machine learning approaches treat each transaction as an independent, identically distributed feature vector. While effective at identifying intratransaction anomalies—unusually large amounts, geographical outliers, or abnormal device signatures—they are structurally blind to the inter-entity relational patterns that define sophisticated modern fraud schemes. A fraud ring typically involves multiple accounts, cards, and devices sharing overlapping linkages across dozens of individually legitimate-looking transactions. Such patterns are encoded exclusively in the topology of a transaction graph, invisible to any tabular model regardless of its complexity. Graph Neural Networks (GNNs) address this fundamental limitation by learning node representations through iterative aggregation of structural neighbourhood information, capturing both node-level features and multihop relational context simultaneously. The key insight exploited in this project is that fraudsters systematically reuse payment cards, device fingerprints, and email domains across multiple transactions—and these reuse patterns create distinctive subgraph topologies directly detectable via message-passing operations. This paper describes the end-to-end development of a production-structured fraud detection system that: (1) converts the IEEE-CIS tabular dataset into a heterogeneous PyTorch Geometric graph with five node types and four edge relation types; (2) trains a HeteroGAT with class-imbalance mitigation; (3) explains individual predictions using GNNExplainer and gradient-based attribution; and (4) deploys findings in an interactive Streamlit dashboard. Systematic comparisons against MLP, GCN, GraphSAGE, and GAT baselines on identical protocols validate each architectural contribution.

To transcend isolated anomaly detection and embrace relational intelligence. By embedding graph-based reasoning into production pipelines, organisations can anticipate evolving fraud strategies, safeguard digital trust, and ensure resilience against adversaries exploiting systemic interconnectivity across financial ecosystems.

A. Problem Statement and Motivation

Given a tabular transaction dataset where each transaction is associated with card, device, email, and address identifiers, the objective is to classify each transaction as either legitimate ($\text{isFraud}=0$) or fraudulent ($\text{isFraud}=1$). The challenge is threefold: first, the dataset exhibits severe class imbalance (3.5% fraud prevalence); second, the fraud signal is distributed across relational entity networks rather than concentrated in individual feature vectors; and third, regulatory interpretability requirements demand transaction-level explanations. This work demonstrates that all three challenges are simultaneously addressed by a properly designed Heterogeneous GNN with integrated explainability.

B. Paper Organisation

Section II reviews background on GNN architectures for fraud detection. Section III describes the IEEE-CIS dataset and preprocessing pipeline. Section IV presents graph construction and the HeteroGAT architecture. Section V details training configuration, imbalance mitigation, and explainability. Section VI reports experimental results and ablation findings. Section VII discusses limitations. Section VIII outlines future directions. Section IX concludes.

II. BACKGROUND AND RELATED WORK

A. Why Graphs Beat Tables for Fraud Detection

The fundamental limitation of tabular fraud detection is its assumption of transactional independence. Each row is processed independently, so any signal embedded in cross-transaction relationships is inaccessible. Two transactions sharing a card (card1) are connected by a same_card edge in our graph, enabling the GNN to propagate the fraud label of a confirmed fraudulent transaction to its card-sharing neighbours across multiple message-passing hops. Table I summarises the key contrasts between tabular ML and graph-based approaches.

TABLE I

Comparison of Tabular ML vs. Graph Neural Networks Along Key Fraud Detection Dimensions

Dimension	Tabular ML (MLP/XGBoost)	Graph Neural Network (HeteroGAT)
Fraud ring detection	Not possible — each row processed independently	Explicit via multi-hop message passing
Card reuse detection	Single feature column (card1)	Shared entity nodes + same_card edges
Device pattern learning	Single column per device attribute	Device node embeddings across all transactions

Dimension	Tabular ML (MLP/XGBoost)	Graph Neural Network (HeteroGAT)
Feature engineering	Manual domain expertise required	Automatic structural pattern learning
Transductive reasoning	None — no cross-row context	2–3-hop neighbourhood aggregation
Explainability	SHAP on input features	GNNExplainer on subgraph + edge masks

B. Graph Convolutional Networks (GCN)

Kipf and Welling (2017) introduced GCN via spectral graph convolution approximated by a first-order Chebyshev polynomial: $H^* = \text{ReLU}(\hat{A}HW)$, where $\hat{A}$ is the degree-normalised adjacency with self-loops. GCN performs symmetric mean aggregation, treating all neighbours equally—a limitation in fraud detection where a single fraudulent neighbour may carry disproportionate signal. GCN is also transductive, making it impractical for production deployment where new transactions arrive continuously.

C. GraphSAGE: Inductive Neighbourhood Sampling

Hamilton et al. (2017) introduced GraphSAGE as an inductive extension to GCN. Rather than operating on the full adjacency matrix, GraphSAGE learns aggregator functions that sample fixed-size neighbourhoods (25 first-hop, 10 second-hop) and concatenate the sampled mean with the current node representation. This inductive formulation allows embedding generation for previously unseen nodes at inference time—essential for real-time fraud scoring where new accounts are created continuously.

D. Graph Attention Networks (GAT)

Velickovic et al. (2018) proposed replacing fixed neighbourhood aggregation with learned attention coefficients: $e_{ij} = \text{LeakyReLU}(a^T[Wh_i \parallel Wh_j])$. Coefficients are normalised via softmax over each node’s neighbourhood, enabling the model to selectively upweight suspicious neighbours. In fraud detection, a transaction sharing a card with one confirmed-fraud transaction should upweight that fraudulent neighbour’s signal—attention achieves exactly this. This project uses GATConv as the base convolution within HeteroConv, with 4 attention heads at intermediate layers and 1 at the final layer.

E. Heterogeneous Graph Neural Networks

Real-world financial graphs are inherently heterogeneous: multiple node types (Transaction, Card, Email, Device, Address) are connected by semantically distinct edge types. Collapsing this heterogeneity into a homogeneous graph loses critical semantic distinctions—a card-sharing connection carries very different fraud signal than an email-domain-sharing connection. This project implements HeteroConv from PyTorch Geometric, applying a separate GATConv operator for each edge type and aggregating results with mean pooling across relation types.

F. Class Imbalance in Fraud Detection

The IEEE-CIS dataset contains only 3.5% fraudulent transactions. Standard cross-entropy loss is dominated by the majority class, causing models to default to predicting all transactions as legitimate and achieving high accuracy but near-zero fraud recall. This project addresses imbalance through class-weighted cross-entropy loss, computing weights inversely proportional to class frequency: $w_{\text{fraud}} = N/(2 \times N_{\text{fraud}}) \approx 14.3$ and $w_{\text{legitimate}} \approx 0.52$. This rebalances gradient contributions without modifying graph structure, which would distort topology-based learning.

G. GNN Explainability

Ying et al. (2019) introduced GNNExplainer, which identifies the subgraph and node features most relevant to a specific prediction by optimising soft masks over edges and features to maximise mutual information with the model’s prediction. This project implements both GNNExplainer (local: per-transaction explanation) and gradient $\times$ input attribution (global: which features drive fraud predictions across all transactions), directly addressing regulatory interpretability requirements.

III. DATASET DESCRIPTION AND PREPROCESSING

A. IEEE-CIS Transaction Fraud Dataset

The IEEE-CIS dataset, released for the Kaggle IEEE Fraud Detection competition in partnership with Vesta Corporation, comprises 590,540 e-commerce transaction records across two files: `train_transaction.csv` and `train_identity.csv`. The transaction file contains 394 features spanning transaction amount, card identifiers (`card1`–`card6`), email domains (`P_emaildomain`, `R_emaildomain`), address codes (`addr1`, `addr2`), and 339 engineered behavioural features (`Vxx`, `Cxx`, `Dxx`, `Mxx` series). The identity file contains 41 additional device and browser features for approximately 140,000 transactions.

Ground-truth fraud labels (`isFraud`) were determined by Vesta Corporation from manual investigation and chargeback records. The dataset exhibits 3.5% positive class prevalence (20,663 fraud cases). The two files are merged on `TransactionID` using a left join, preserving all 590,540 transaction records while incorporating identity features where available.

B. Preprocessing Pipeline (`utils/preprocessing.py`)

The complete preprocessing pipeline is implemented in `utils/preprocessing.py` and consists of five sequential stages. The `preprocess_pipeline()` function orchestrates all stages and returns a processed `DataFrame`, fitted `LabelEncoders`, and a fitted `StandardScaler` for consistent inference-time transformation.

Stage 1 — Load and Merge

The `load_and_merge()` function reads both CSVs with `pd.read_csv()` and performs a left join on `TransactionID`, producing a unified `DataFrame` of shape (590,540, 435). The left join fills identity columns with

NaN for the ~450,000 transactions without identity records.

Stage 2 — Missing Value Handling

The `handle_missing()` function applies a two-tier strategy. Any column with more than 90% missing values is dropped entirely, excluding the `isFraud` target and `TransactionID`. Remaining numerical columns are imputed with column-wise median, and categorical columns are filled with the sentinel value ‘UNKNOWN’, subsequently label-encoded as a valid category.

Stage 3 — Categorical Encoding

The `encode_categoricals()` function applies `scikit-learn` `LabelEncoder` to all object-type columns (`card4`, `card6`, `P_emaildomain`, `R_emaildomain`, `DeviceType`, `DeviceInfo`, `id_xx`). Each fitted encoder is stored in a dictionary and serialised with `joblib` for inference-time inverse transformation.

Stage 4 — Numerical Normalisation

A `log1p` transformation is applied to `TransactionAmt` to correct its heavy right skew before scaling. All numerical columns are then standardised using `StandardScaler` to zero mean and unit variance.

Stage 5 — Class Weight Computation

Class weights are computed as $w_c = N/(2 \times N_c)$, yielding $w_{\text{legitimate}} \approx 0.519$ and $w_{\text{fraud}} \approx 14.29$. These weights are converted to a CUDA-compatible tensor for use in the weighted `CrossEntropyLoss` during training.

IV. GRAPH CONSTRUCTION (UTILS/GRAPH_BUILDER.PY)

A. Graph Architecture Philosophy

The `graph_builder.py` module is the architectural centrepiece of this system, operationalising the core insight that fraudsters systematically reuse payment instruments. Its design is explicitly motivated by three detection strategies:

- Fraud ring exposure: confirmed fraudulent nodes propagate fraud signal to their shared-card neighbours across multiple message-passing hops.
- Entity-level behavioural embedding: Device, email, and address nodes accumulate aggregated features from all transactions they participate in, learning entity-level behaviour profiles.
- Multi-hop reasoning: A 3-layer GNN can detect an entire fraud ring in a single forward pass through the pattern $\text{Txn_A} \rightarrow \text{Card_X} \leftarrow \text{Txn_B} \rightarrow \text{Device_Y} \leftarrow \text{Txn_C}$.

B. Node Types

The `build_graph()` function constructs a `PyTorch` `Geometric HeteroData` object with five node types, summarised in Table II. Entity node features are computed as the mean of all connected transaction feature vectors, encoding the aggregate behavioural profile of transactions sharing that entity.

TABLE II
Node Types in the Heterogeneous Transaction Graph

Node Type	Source Column	Count	Feature Construction
transaction	One per row	590,540	Standardised numerical features from preprocessing
card	card1	~6,170	Mean of all connected transaction feature vectors
email	P_emaildomain	~59	Mean of all connected transaction feature vectors
device	DeviceType	3	Mean of all connected transaction feature vectors
address	addr1	~332	Mean of all connected transaction feature vectors

C. Edge Types and Fraud Ring Detection

Eight edge types are constructed: four forward relations (uses_card, uses_email, uses_device, uses_addr) and four reverse relations enabling bidirectional message passing. A ninth edge type, same_card, connects transaction nodes sharing the same card1 value—the primary fraud ring detection edge. Transactions are grouped by card1 and all pairwise edges are created within groups of 2–50 (a cap of 50 prevents $O(n^2)$ edge explosion on popular legitimate cards). This transaction-to-transaction edge enables a fraudulent transaction to contaminate its card-sharing neighbours even when their individual features appear legitimate.

D. Graph Statistics on IEEE-CIS

Table III summarises key statistics of the constructed heterogeneous transaction graph on the full IEEE-CIS dataset.

TABLE III
Key Statistics of the Constructed Heterogeneous Transaction Graph

Graph Property	Value
Total transaction nodes	590,540
Total card nodes	~6,170
Total email nodes	~59
Total device nodes	3
Total address nodes	~332
Transaction→Card edges (uses_card)	590,540
Transaction→Email edges (uses_email)	590,540
Transaction→Device edges (uses_device)	590,540
Transaction→Address edges (uses_addr)	590,540
Transaction↔Transaction edges (same_card)	~3.2M

Graph Property	Value
Class-weight (fraud)	≈ 14.29
Class-weight (legitimate)	≈ 0.519

V. MODEL ARCHITECTURE AND TRAINING PIPELINE

A. FraudGNN: Heterogeneous Graph Attention Network

The core model (models/gnn_model.py: FraudGNN) is a three-layer Heterogeneous Graph Attention Network. The architecture was selected over GraphSAGE for three reasons specific to fraud detection: (1) attention weights are interpretable, revealing which neighbours drive each prediction; (2) fraudulent nodes have abnormal neighbourhood patterns that attention mechanisms amplify; and (3) multi-head attention reduces variance from noisy benign neighbours.

Input Projection Layer

A LazyLinear layer per node type projects raw features to the shared hidden dimension of 128. LazyLinear defers input dimension inference to the first forward pass, accommodating feature counts that vary based on preprocessing outcomes.

HeteroConv Layers

Three HeteroConv layers are stacked, each containing a separate GATConv operator for each of the eight edge types. At intermediate layers, GATConv uses 4 attention heads with concat=False and Dropout ($p=0.3$). At the final layer, 1 attention head is used for clean classification signal. Each HeteroConv aggregates multi-relation outputs with mean pooling.

Batch Normalisation and Regularisation

BatchNorm1d is applied per node type after each intermediate HeteroConv layer, followed by ReLU activation and Dropout ($p=0.3$). Gradient clipping ($\max_norm=1.0$) is applied during training to prevent exploding gradients from sparse high-weight fraud nodes.

Classification Head

The classification head operates exclusively on transaction node embeddings extracted from the final HeteroConv layer. A two-layer MLP [Linear(out_channels, 64), ReLU, Dropout(0.3), Linear(64, 2)] produces logits for binary fraud classification.

B. FraudGraphSAGE: Alternative Architecture

An alternative implementation, FraudGraphSAGE, is provided for deployment contexts where graph size exceeds one million edges and GAT’s $O(|E|)$ attention computation becomes prohibitive. FraudGraphSAGE uses SAGEConv layers with L2 normalisation inside HeteroConv, trading interpretable attention weights for faster training and lower memory footprint.

C. Training Pipeline (train.py)

The complete training pipeline in train.py accepts transaction file path, identity file path, epoch count, learning rate, hidden dimension, attention heads, layer count, dropout rate, and model type (gat or sage) as command-line arguments.

Train/Validation/Test Split

The make_masks() function applies a stratified 70/15/15 split using sklearn.model_selection.train_test_split with fixed random seed (42). Boolean masks are assigned to data['transaction'].train_mask, .val_mask, and .test_mask. The test set is reserved exclusively for final evaluation.

Optimisation

Training uses Adam (lr=0.001, weight_decay=1e−5) with ReduceLROnPlateau scheduling (patience=5, factor=0.5). Early stopping tracks the best validation AUC-ROC and saves model.state_dict() to models/saved/best_model.pt. The weighted CrossEntropyLoss(weight=[w0,w1]) is computed exclusively on train_mask nodes.

Evaluation and Serialisation

After training, the best checkpoint is reloaded and evaluated on the test mask. Training curves, confusion matrix, ROC curve, and all model artifacts (encoders.pkl, scaler.pkl, model_meta.pkl) are serialised for dashboard loading.

D. Explainability Pipeline (models/explainability.py)

Three complementary explainability methods are implemented. First, compute_gradient_feature_importance() computes gradient × input attribution for 500 test transactions, consistently identifying TransactionAmt, card1 frequency, and C-series features as top fraud predictors. Second, explain_single_transaction() applies GNNExplainer (200 optimisation epochs) to generate node and edge importance masks for any individual transaction, answering “Why was this specific transaction flagged?” Third, detect_fraud_clusters() applies Louvain community detection to flag communities where the mean predicted fraud probability exceeds 0.5 as suspected fraud rings.

VI. EXPERIMENTAL RESULTS

A. Comparative Performance

All five models are evaluated on the identical IEEE-CIS test partition using the same preprocessing pipeline, class-weighted loss, and evaluation protocol. Performance is reported as the mean across 5 independent random-seed runs. Results are summarised in Table IV.

TABLE IV

Comparative Classification Performance on the IEEE-CIS Test Set

Model	Training Mode	AUC - ROC	Precision	Recall	F1-Score
MLP (Baseline)	Tabular features	0.891	0.74	0.61	0.67
GCN	Homogeneous graph	0.921	0.79	0.68	0.73
GraphSAGE	Inductive GNN	0.943	0.83	0.72	0.77
GAT	Attention GNN	0.951	0.85	0.76	0.80
HeteroGAT (Ours)	Heterogeneous GAT	0.971	0.89	0.83	0.86

HeteroGAT achieves the highest AUC-ROC of 0.971 and fraud-class F1-Score of 0.86, representing an 8.0-point AUC improvement over the MLP tabular baseline and a 2.0-point improvement over the strongest homogeneous baseline (GAT at 0.951). The monotonically improving performance from MLP → GCN → GraphSAGE → GAT → HeteroGAT validates the incremental value of: (1) graph structure over tabular features, (2) inductive vs. transductive formulation, (3) attention over mean aggregation, and (4) heterogeneous type-specific transformations.

B. Ablation Study

Table V presents a systematic ablation quantifying the contribution of each architectural component. The most critical single contribution is graph structure itself (8.0 AUC points vs. tabular MLP), confirming that relational information is the dominant performance driver. Attention contributes 3.0 AUC points over mean aggregation and heterogeneous type-awareness contributes an additional 2.0 points. Critically, removing class-weighted loss collapses fraud recall from 0.83 to 0.61 while having minimal effect on overall AUC—confirming that imbalance handling is essential for practical fraud detection. The same_card transaction-to-transaction edges contribute 1.3 AUC points, directly encoding fraud ring propagation.

TABLE V

Ablation Study: Contribution of Each HeteroGAT Component

Ablation Condition	AUC-ROC	Fraud Recall	Δ AUC vs Full Model
Full HeteroGAT (all components)	0.971	0.83	—
Remove type-specific projections (homogeneous)	0.951	0.76	−2.0 pts
Remove attention (mean aggregation)	0.921	0.68	−5.0 pts
Remove class-weighted loss (uniform CE)	0.962	0.61	−0.9 pts; −22 pts recall
Remove graph structure (MLP only)	0.891	0.61	−8.0 pts
Remove same_card edges (no tx-tx edges)	0.958	0.74	−1.3 pts

C. Computational Efficiency

Table VI presents the computational profiles of all five models on the full IEEE-CIS dataset using an NVIDIA A100 40GB GPU. HeteroGAT’s 12.3ms per-batch inference latency remains within acceptable bounds for real-time fraud scoring systems (typically 50–100ms decision windows). GraphSAGE’s inductive formulation and lower memory footprint make it the preferred architecture for edge deployment where model size is constrained.

TABLE VI

Computational Profile on IEEE-CIS (NVIDIA A100 40GB GPU)

Model	Params (M)	Train Time (s/epoch)	Inference (ms/batch)	GPU Memory (GB)
MLP Baseline	0.8	12	1.2	0.4
GCN	1.2	31	3.7	1.1
GraphSAGE	2.1	48	5.4	1.8
GAT	3.4	76	9.1	2.9
HeteroGAT (Ours)	4.7	104	12.3	3.8

VII. STREAMLIT DASHBOARD (APP.PY)

A. Dashboard Architecture

The interactive dashboard is implemented in app.py using Streamlit 1.28 with a dark theme ('0E1117' background). It loads pre-computed artifacts from models/saved/ via st.cache_data and provides five interactive pages via sidebar navigation.

B. Dashboard Pages

- Overview Page: Four KPI metrics (Test AUC-ROC, Fraud Count, Fraud Rate, Best Validation AUC) and a fraud probability distribution histogram via Plotly.
- Transaction Explorer: Interactive table filtered by fraud probability threshold, with CSS-coded badges (red=fraud, green=safe).
- Graph Visualisation: PyVis-generated interactive HTML graph embedded via st.components.v1.html, showing fraud clusters with colour-coded node types.
- Model Performance: Plotly ROC curve with AUC annotation, confusion matrix heatmap, and transaction amount distribution by fraud label.
- Feature Importance: Horizontal bar chart of gradient $\times$ input attribution scores for the top 20 features.

C. Quick Start

The system is launched after training with the following commands:

```
python train.py --transaction data/train_transaction.csv \
--identity data/train_identity.csv --epochs 50 --model
gat
streamlit run app.py
```

The system is also Dockerised and deployable to Streamlit Cloud by committing models/saved/ artifacts to the repository.

VIII. DISCUSSION

A. Structural Advantages of Graph-Based Fraud Detection

The 8.0 AUC-ROC point advantage of HeteroGAT over the MLP baseline translates directly into financial impact. Assuming a mean fraud value of USD 200 and daily volume of 100,000 transactions, a 22-point recall improvement (0.83 vs. 0.61) corresponds to approximately USD 4.4M in additional fraud prevented per day. This quantification motivates the additional computational cost of graph construction and HeteroConv-based inference relative to tabular baselines.

B. The Role of Entity Nodes as Behaviour Profiles

A distinctive architectural choice in this project is constructing entity nodes (Card, Email, Device, Address) with features computed as the mean of connected transaction features. This transforms entity nodes into learnable aggregated behaviour profiles: a card node’s embedding encodes the statistical footprint of all transactions that used that card. After training, the GNN’s entity node embeddings effectively learn “risky card profiles” and “safe email domain profiles” without requiring explicit entity-level feature engineering.

C. Limitations

Several practical limitations qualify the reported results. First, the system is trained on a static graph

snapshot; production deployment requires incremental graph updates as new transactions arrive, which the current batch training paradigm does not support without complete retraining. Second, the same card edge construction caps co-occurrence groups at 50 to prevent $O(n^2)$ edge explosion, which may limit ring detection for large legitimate card-sharing networks. Third, GNNExplainer’s optimisation-based explanation generation (200 epochs per explanation) is computationally expensive for high-throughput real-time scoring without pre-computation. Fourth, the system operates on the IEEE-CIS benchmark; production fraud data is substantially larger, noisier, and subject to adversarial adaptation.

IX. FUTURE WORK

A. Temporal Graph Neural Networks

The current architecture treats the transaction graph as a static snapshot, ignoring the temporal ordering of transactions. Integrating Temporal Graph Networks (TGN) or EvolveGCN would enable detection of velocity-based fraud (multiple high-value transactions in a short window) and evolving fraud strategies that adapt over time.

B. Streaming Graph Inference

GraphSAGE’s inductive formulation (implemented as FraudGraphSAGE) provides the foundation for incremental embedding updates without full graph retraining. A streaming inference pipeline would precompute and cache entity node embeddings, enabling sub-10ms latency for real-time fraud scoring.

C. Federated Graph Learning

Financial institutions cannot share raw transaction data due to GDPR and India’s DPDP Act constraints. Federated GNN training protocols, in which multiple institutions exchange gradients rather than data, would enable fraud ring detection that spans institutional boundaries—the most sophisticated fraud operations explicitly exploit cross-institution blind spots.

D. Adversarial Robustness

Sophisticated fraud actors will eventually attempt to poison the training graph by injecting transactions designed to “whiten” their card nodes’ behavioural profiles. Systematic evaluation of HeteroGAT robustness to node feature perturbation, edge injection, and label-flipping attacks is essential before production deployment.

E. Production Deployment via FastAPI

Packaging the trained HeteroGAT as a REST API endpoint (FastAPI + Uvicorn) with serialised PyTorch Geometric HeteroData objects and streaming preprocessing middleware would enable direct integration into banking transaction processing pipelines.

X. CONCLUSION

This paper has presented the complete design, implementation, and evaluation of a Heterogeneous Graph Attention Network (HeteroGAT) for financial fraud

detection on the IEEE-CIS Transaction Fraud benchmark. The system transforms a 590,540-record tabular dataset into a multi-relational heterogeneous graph with five node types and nine edge relation types, enabling the GNN to detect fraud rings through message propagation across shared card, email, device, and address entities.

The HeteroGAT architecture—three layers of HeteroConv-wrapped GATConv with type-specific projections, multi-head attention, Batch Normalisation, Dropout, and class-weighted cross-entropy loss—achieves an AUC-ROC of 0.971 and fraud-class F1-Score of 0.86, outperforming tabular MLP, GCN, GraphSAGE, and homogeneous GAT baselines by 8.0, 5.0, 2.8, and 2.0 AUC points respectively. Ablation studies isolate each contribution: graph structure contributes 8.0 points, attention 3.0 points, heterogeneous type-awareness 2.0 points, and class-weighted loss 22 recall percentage points.

The project further contributes a production-structured codebase (modular preprocessing, graph builder, model registry, explainability module, Streamlit dashboard) and a complete deployment pathway (Docker, Streamlit Cloud). The GNNExplainer-based explainability pipeline directly addresses regulatory interpretability requirements by providing transaction-level subgraph explanations. Together, these contributions demonstrate that heterogeneous graph learning is both technically superior to and practically deployable as a replacement for tabular machine learning in production fraud detection systems.

ACKNOWLEDGEMENTS

The authors gratefully acknowledge the guidance of Dr. Pradeep Upadhyay, Assistant Professor, School of Computer Science and Engineering, IILM University, whose mentorship shaped the methodological rigour of this research. The authors also acknowledge the PyTorch Geometric development team (Fey & Lenssen, 2019) for the HeteroConv and GATConv implementations that form the backbone of this work, and Vesta Corporation and Kaggle for making the IEEE-CIS fraud detection benchmark publicly available. All computational experiments were conducted using Google Colab A100 GPU infrastructure.

REFERENCES

- [1] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in Proc. ICLR 2017.
- [2] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in Proc. NeurIPS 2017, pp. 1024–1034.
- [3] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, “Graph attention networks,” in Proc. ICLR 2018.
- [4] J. Gilmer et al., “Neural message passing for quantum chemistry,” in Proc. ICML 2017, pp. 1263–1272.
- [5] R. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec, “GNNExplainer: Generating explanations for graph neural networks,” in Proc. NeurIPS 2019.

-
- [6] Y. Dou et al., “Enhancing graph neural network-based fraud detection via sequential neural networks,” in Proc. CIKM 2020, pp. 315–324.
 - [7] Z. Liu et al., “Heterogeneous graph neural networks for malicious account detection,” in Proc. CIKM 2018, pp. 2077–2085.
 - [8] X. Wang et al., “Heterogeneous graph attention network,” in Proc. WWW 2019, pp. 2022–2032.
 - [9] S. Yun et al., “Graph transformer networks,” in Proc. NeurIPS 2019.
 - [10] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollar, “Focal loss for dense object detection,” in Proc. ICCV 2017, pp. 2980–2988.
 - [11] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic minority over-sampling technique,” *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002.
 - [12] M. Weber et al., “Anti-money laundering in Bitcoin: Experimenting with graph convolutional networks for financial forensics,” in KDD Workshop on Anomaly Detection in Finance, 2019.
 - [13] M. Fey and J. E. Lenssen, “Fast graph representation learning with PyTorch Geometric,” in ICLR Workshop on Representation Learning on Graphs and Manifolds, 2019.
 - [14] ACFE, “Report to the Nations: 2024 Global Study on Occupational Fraud and Abuse,” Association of Certified Fraud Examiners, 2024.
 - [15] X. Zhang, Z. Liu, J. Li, D. Wang, and J. Tang, “Beyond homophily: Structural graph similarity for fraud detection,” in Proc. AAAI 2022.
-